

# ค้นสลากสิบล้านใบ และแจกแต่ละใบให้ได้เพียงครั้งเดียว

ค้นสลาก 6 หลักจำนวน 10,000,000 ใบ ด้วย pattern ที่มี wildcard แล้วแจกให้ผู้ใช้ที่เข้ามาพร้อมกัน โดยต้องไม่มีตัวใบใดไปถึงคนสองคน ตัวเลขทุกตัวในเอกสารนี้เป็นค่าที่วัดจริงจากระบบที่สร้างขึ้นมา และทดสอบภายใต้ resource limit ที่บังคับใช้จริง ไม่ใช่การประมาณ

# ข้อมูลกำกับเอกสาร

|                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ชื่อเอกสาร<br>Lottery Search System — Design Document                                                                                                                                     |
| ผู้เขียน<br>Natthawat Narin, Senior Backend Developer                                                                                                                                     |
| สถานะ<br>ฉบับสมบูรณ์ — ส่งเพื่อรีวิว                                                                                                                                                      |
| เวอร์ชัน<br>1.0                                                                                                                                                                           |
| วันที่<br>2026-09-05                                                                                                                                                                      |
| ผู้รีวิว<br>—                                                                                                                                                                             |
| REPOSITORY<br><a href="#">github.com/nthw-dev/challenge-lottery-search-system</a> — เอกสารฉบับนี้ บันทึกการตัดสินใจ ผลวิเคราะห์ฉบับเต็ม และ evidence bundle ทุกชุดที่อยู่เบื้องหลังตัวเลข |
| PROOF OF CONCEPT<br><a href="#">github.com/nthw-dev/poc-lottery-search-system</a> — solution architecture, data structures, algorithms และ loadtest ที่ให้ตัวเลขทุกตัวในเอกสารนี้         |
| ตอบโจทย์<br><code>challenge-lottery-search-system.md</code>                                                                                                                               |
| บันทึกการตัดสินใจ<br><code>adr/</code> — architecture decision record 8 ใบ คูสารบัญในภาคผนวก A                                                                                            |

ตัวเลขทุกตัวในเอกสารนี้เป็นค่าที่วัดจริง ไม่ใช่การประมาณ มีการสร้าง proof of concept และทดสอบภายใต้ resource limit ที่บังคับใช้จริงบน Kubernetes เพื่อให้ได้ตัวเลขเหล่านี้โดยเฉพาะ ๕8 เป็นสารบัญของหลักฐานดิบ และ ๕9 ระบุสิ่งที่ยังไม่ได้พิสูจน์

## พื้นที่ค้นหาจริงคือหนึ่งล้าน ไม่ใช่สิบล้าน

โจทย์คือ ค้นหา 6 หลักจำนวน 10,000,000 ใบ ด้วย pattern 6 ตัวอักษรที่มี wildcard \* แล้วแจกสลากที่เข้าเงื่อนไขให้ผู้ใช้ที่เข้ามาพร้อมกัน โดยต้องไม่มีสลากใบใดไปถึงคนสองคน สิ่งที่ต้องส่งคือ การออกแบบ พร้อมเทคโนโลยี storage ที่แนะนำ กลยุทธ์ index สำหรับ wildcard การวิเคราะห์ประสิทธิภาพ และกลยุทธ์ด้าน concurrency ที่ป้องกันการแจกซ้ำ

### 2.1 กฎเกณฑ์สำคัญที่การออกแบบทั้งหมดตั้งอยู่

สลาก 6 หลักมีค่าที่เป็นไปได้เพียง  $10^6 = 1,000,000$  ค่า เมื่อมีสลาก 10 ล้านใบ จึงเฉลี่ยได้ประมาณ 10 ใบต่อหนึ่งเลข

ข้อเท็จจริงนี้เปลี่ยนมุมมองของโจทย์ทั้งหมด การมองแบบตรงไปตรงมาคือ "ค้นข้อมูลสิบล้านแถว" แต่การมองที่ถูกต้องคือ "ตัดสินใจว่าเลขไหนบ้างในหนึ่งล้านเลขที่เข้าเงื่อนไข แล้วค่อยดึงสลากของเลขเหล่านั้น" พื้นที่ค้นหาแบบที่สองเล็กกว่าสิบล้านเท่า มีขนาดคงที่ไม่ว่าจะมีสลากกี่ใบ และเล็กพอที่จะอยู่ใน cache ได้ตลอดเวลา การตัดสินใจสำคัญทุกข้อในเอกสารนี้สืบเนื่องมาจากจุดนี้

อีกเรื่องที่ต้องเข้าใจก่อนอ่านตัวเลขประสิทธิภาพใดๆ คือขนาดของผลลัพธ์ต่างกันได้ถึงหกอันดับตามจำนวน wildcard

| PATTERN | WILDCARD | เลขที่เข้าเงื่อนไข | สลากที่เข้าเงื่อนไข |
|---------|----------|--------------------|---------------------|
| 123456  | 0        | 1                  | ~10                 |
| 123***  | 3        | 1,000              | ~10,000             |
| ****23  | 4        | 10,000             | ~100,000            |
| **3***  | 5        | 100,000            | ~1,000,000          |

| PATTERN | WILDCARD | เลขที่เข้าเงื่อนไข | สลาที่เข้าเงื่อนไข |
|---------|----------|--------------------|--------------------|
| *****   | 6        | 1,000,000          | 10,000,000         |

ระบบที่เร็วเฉพาะ pattern แคบ หรือเร็วเฉพาะ pattern กว้าง ยังไม่ถือว่าแก้โจทย์ได้ การ  
รายงานประสิทธิภาพจึงต้องแยกตามจำนวน wildcard ซึ่งเป็นวิธีที่ §6.2 ใช้

wildcard อยู่ตำแหน่งใดก็ได้ ทำให้ B-tree ปกติบนคอลัมน์เลขสลาที่ใช้ได้เฉพาะตอนที่หลัก  
หน้าถูกล็อกไว้ กรณี \*\*\*\*\*23 จะใช้ไม่ได้เลย ข้อเท็จจริงข้อเดียวนี้คือแก่นทางเทคนิคของโจทย์

03 เป้าหมายและสิ่งที่ไม่ทำ

เป้าหมายที่ทดสอบได้ 4 ข้อ ผ่านทั้งหมด และ 6 สิ่งที่ตั้งใจไม่ทำ

3.1 เป้าหมาย

ทุกเป้าหมายถูกตั้งเป็นสมมติฐานที่ทดสอบได้ก่อนสร้าง proof of concept และแต่ละข้อจับคู่  
กับผลที่วัดได้จริง

| เป้าหมาย                                                                                      | เกณท์                                                              | ที่วัดได้                                                                                 | ผล          |
|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------------|
| G1 คำน wildcard ทุกตำแหน่งที่ 10M rows ภายใต้ API pod 500m CPU / 128Mi                        | p95 < 100 ms ทุกคลาส wildcard ที่ผู้ใช้ 100 คน พร้อมกัน            | p95 55.8 ms แบนราบทุกคลาส 5,346 req/s                                                     | ผ่าน (§6.2) |
| G2 ผู้ใช้จำนวนมากค้น pattern เดียวกัน พร้อมกัน ไม่มีใครได้ตัวใบเดียวกัน                       | ซ้ำ 0 ภายใต้การแย่งกัน 200 ทาง                                     | จอง 351,481 ใบ ซ้ำ 0 ledger ไม่ตรงกับสถานะ 0                                              | ผ่าน (§8.1) |
| G3 กลยุทธ์ index ที่เลือกเร็วกว่า baseline แบบ LIKE อย่างมีนัยสำคัญ ที่ต้นทุนพื้นที่ยอมรับได้ | ≥ 10 เท่าใน pattern ที่ยากที่สุด ขนาด index เทียบเคียงทางเลือกอื่น | สี่ order of magnitude ใน pattern ระบุครบ index 192 MB เทียบ 397 MB ของทางเลือกแบบรายหลัก | ผ่าน (§6.2) |
| G4 เสถียรภายใต้โหลดผสมต่อเนื่อง ภายใน memory limit ของ pod                                    | ไม่โดน OOM kill memory ต่ำกว่า 128Mi มากตลอด soak 10 นาที          | 743,994 request error 0 restart 0 สูงสุด 27 MiB                                           | ผ่าน (§7.3) |

## 3.2 สิ่งที่ตั้งใจไม่ทำ

อยู่นอกขอบเขตโดยตั้งใจ เพื่อพิสูจน์ว่า database ตัวเดียวเพียงพอ ก่อนจะเพิ่มอะไรเข้าไป

- Authentication และ authorization
- Payment ระบบออกรางวัล และระบบงวด
- การขยาย database แนวนอน replication และ read replica
- Caching layer ทุกชนิด — จงใจไม่ใช่ Redis
- Observability stack ระดับ production (Prometheus, Grafana, tracing)
- CI/CD pipeline

### 04 ภาพรวม

## API ไม่เก็บ state หนึ่งตัว database หนึ่งตัว ตารางสามตาราง

STORAGE ที่แนะนำ

**PostgreSQL 16** ตัวเดียว ไม่มี cache ไม่มี lock service ไม่มี search engine

วิธีค้นหา

หาเลขที่เข้า pattern จากตาราง **1,000,000** แถว ก่อน แล้วค่อย join เข้าตารางสลาก

วิธีแจกของ

**FOR UPDATE SKIP LOCKED** ในคำสั่งเดียว คู่กับ ledger แบบ append-only

ผลวัดที่ 10M ROWS

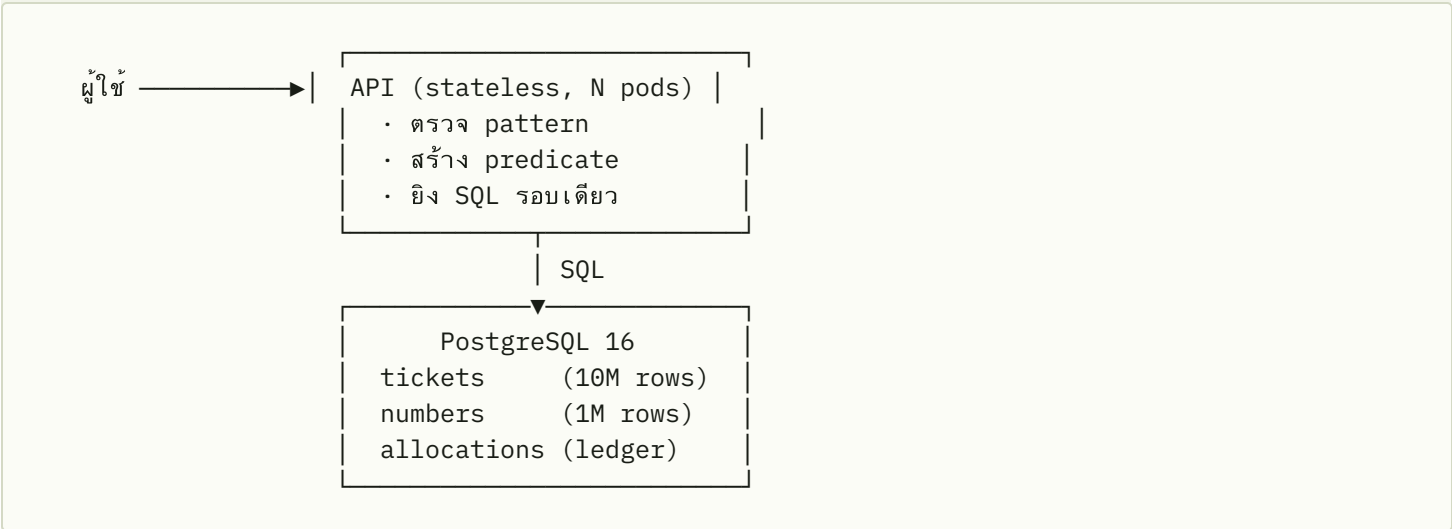
**p95 56 ms** ที่ 5,346 req/s บน pod ขนาด 500m CPU และจอง **351,481** ไบต์ ใช้ **0**

การออกแบบคือ API ที่ไม่เก็บ state หนึ่งตัว อยู่หน้า database PostgreSQL หนึ่งตัวที่มีสามตาราง การค้นหาจะตัดสินใจเลขไหนบ้างในหนึ่งล้านเลขที่เข้า pattern โดยใช้ตารางเล็กที่มี index รายหลัก แล้วจึงค่อยแตะตารางสลากสิบล้านแถว การจองคือ SQL คำสั่งเดียวที่

การล็อกแถวทำให้ผู้ใช้สองคน ได้ตัวใบเดียวกันไม่ได้ และผลถูกบันทึกลง ledger ใน transaction เดียวกัน ไม่มีอย่างอื่นอีกเลย ไม่มี cache ไม่มี lock service ไม่มี search engine วัฏบน PostgreSQL 16 และแนะนำให้ใช้ 17 (§6.4)

# สถาปัตยกรรม โครงสร้างข้อมูล การค้นหา และ concurrency

## 5.1 สถาปัตยกรรมและ API



ตั้งใจไม่ให้มีอย่างอื่นอีกเลย ไม่มี cache ไม่มี queue ไม่มี lock service ไม่มี search engine ทุก component ที่ไม่มีอยู่ ย่อมพังไม่ได้ ย่อมไม่มีทางข้อมูลไม่ตรงกับ database และไม่ต้องมีคนดูแล ส่วนการรับประกันว่าจะไม่แจกซ้ำนั้น database ทำให้เองด้วยการล็อกระดับแถว เหตุผลปกติที่คนจะใส่ Redis เข้ามาจึงหายไป

API ไม่เก็บ state ใดๆ ซึ่งเป็นเหตุผลที่ทำให้การขยายแนวนอนเป็นคำตอบของเรื่อง throughput

| ENDPOINT          | หน้าที่                                               |
|-------------------|-------------------------------------------------------|
| POST /v1/search   | ดูผลอย่างเดียว ไม่จองอะไร จำกัดผลลัพธ์ที่ 100         |
| POST /v1/allocate | จองสลากที่เข้าเงื่อนไขให้ผู้ใช้แบบ atomic จำกัดที่ 20 |

| ENDPOINT         | หน้าที่                                                           |
|------------------|-------------------------------------------------------------------|
| POST /v1/release | คืนสลากที่จองไว้กลับเข้าระบบ                                      |
| GET /healthz     | liveness ตรวจสอบว่า process ยังอยู่ ไม่แตะ database ได้ขาด (§7.3) |
| GET /readyz      | readiness ping database                                           |

pattern ต้องตรงกับ `^[0-9*]{6}$` และถูกปฏิเสธที่ชั้น handler ก่อนแตะ database ซึ่งเป็นทั้งกฎด้านความถูกต้อง และเป็นเหตุผลที่ตัวสร้าง predicate ใส่ตัวเลขลงใน SQL ได้อย่างปลอดภัย ส่วนจำนวนผลลัพธ์เป็นค่าประมาณจากสูตร  $10^{\text{wildcard}} \times \text{สลากต่อเลข}$  ไม่ใช่การนับจริง เพราะการ `COUNT(*)` จริงสำหรับ `**3**` ต้องอ่านเป็นล้านแถวเพื่อให้ได้ตัวเลขที่ไม่มีใครต้องการความแม่นยำ

## 5.2 โครงสร้างข้อมูล

```
CREATE TABLE tickets (  
  id          bigint    PRIMARY KEY,  
  number      int       NOT NULL,          -- 0..999999  
  status      smallint  NOT NULL DEFAULT 0, -- 0=ว่าง, 1=จองแล้ว, 2=ขายแล้ว  
  reserved_by int,  
  reserved_at timestampz  
);  
  
CREATE TABLE numbers (  
  n int PRIMARY KEY,          -- พื้นที่ค้นหา 1 ล้านแถว  
                                -- 0..999999  
  d1 smallint NOT NULL, d2 smallint NOT NULL, d3 smallint NOT NULL,  
  d4 smallint NOT NULL, d5 smallint NOT NULL, d6 smallint NOT NULL  
);  
  
CREATE TABLE allocations (  
                                -- ledger แบบเขียนอย่างเดียว  
  ticket_id bigint    NOT NULL,  
  user_id   int       NOT NULL,  
  at        timestampz NOT NULL DEFAULT now()  
);
```

| การตัดสินใจ                                                           | เหตุผล                                                                                                                                               |
|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>number</code> เป็น <code>int</code> ไม่ใช่ <code>char(6)</code> | ประหยัดราว 60 MB ที่ 10M rows เปรียบเทียบเร็วกว่า และดึงหลักด้วยเลขคณิตได้โดยไม่ต้องแปลง type ส่วนการเติมศูนย์หน้าเป็นเรื่องของการแสดงผล ทำตอนส่งออก |

| การตัดสินใจ                                    | เหตุผล                                                                                                            |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| ไม่เก็บหลักตัวเลขไว้บน <code>tickets</code>    | ถ้าใช้ generated column หกคอลัมน์จะกิน heap เพิ่มราว 120 MB จึงย้ายไปไว้บนตาราง 1 ล้านแถวแทน ซึ่งต้นทุนต่ำกว่ามาก |
| <code>status</code> เป็น <code>smallint</code> | เล็กและเปรียบเทียบเร็วกว่า enum หรือ text และเพียงพอสำหรับสามสถานะ                                                |
| แยกตาราง <code>allocations</code> ออกมา        | ทำให้การรับประกันว่าไม่แจกซ้ำ <b>ตรวจสอบได้ด้วย query เดียว</b> แทนที่จะต้องอนุมานเอา นี่คือหลักฐานที่ §5.4 ใช้   |

`numbers` เป็นตาราง lookup แบบคงที่ สร้างหนึ่งล้านแถวครั้งเดียวแล้วไม่เขียนอีกเลย และเป็นเหตุผลที่ระบบรับสลากรับเพิ่มได้โดยที่ต้นทุนการค้นหาไม่เปลี่ยน

### 5.3 อัลกอริทึมการค้นหาและ index

สำหรับ pattern อย่าง `1*3**6` ขั้นตอนคือ

1. **ตรวจ pattern** แล้วแยกว่าตำแหน่งไหนเป็นตัวเลขคงที่ ตำแหน่งไหนเป็น wildcard
2. **หาเลขที่เข้าเงื่อนไข** จากตาราง `numbers` โดยมีสองกรณีที่ถูกกันไว้เป็นพิเศษ เพราะยุบลงเป็นอะไรที่ถูกกว่าการค้นหา index ได้ คือ **หลักหน้าคงที่ติดกัน** (`123***`) จะกลายเป็นช่วงบน primary key `n BETWEEN 123000 AND 123999` และ **ระบุครบทั้งหกหลัก** (`123456`) จะข้ามตาราง `numbers` ไปเลยเป็น `number = 123456` กรณีอื่นใช้ B-tree ของแต่ละหลัก
3. **ดึงสลากร** โดย join เลขเหล่านั้นเข้ากับ `tickets(number)` ผ่าน partial index ที่จำกัดไว้ที่ `status = 0`
4. **หยุดทันทีที่พอ** โดยดัน `LIMIT` ลงไปถึงขั้น index scan

```
-- 123*** → อ่านเป็นช่วงบน primary key ของตาราง 1 ล้านแถว
SELECT id, number FROM tickets
WHERE status = 0
      AND number IN (SELECT n FROM numbers WHERE n BETWEEN 123000 AND 123999)
LIMIT 20;

-- ****23 → ใช้ index ของแต่ละหลักบนตาราง 1 ล้านแถว
SELECT id, number FROM tickets
WHERE status = 0
      AND number IN (SELECT n FROM numbers WHERE d5 = 2 AND d6 = 3)
LIMIT 20;
```



รวมทั้งหมดมี 7 index คือ index ของหลักบน `numbers` ขนาดราว 6.8 MB ต่อตัว บวกกับ partial B-tree บน `tickets(number)` รวมเป็น **192 MB** ที่สลาก 10 ล้านใบ ส่วนที่อยู่บน `numbers` มีขนาดคงที่ตลอดไป มีเพียง index บนตารางสลากที่โตตามจำนวนสลาก และ เพราะเป็น partial บน `status = 0` มันจึงเล็กลงเมื่อของถูกขายไปเรื่อยๆ

สองกฎที่ได้มาจากการวัดจริง

ห้ามใส่ **ORDER BY** ในการค้นหา คำสั่งที่ดูปกติอย่าง `ORDER BY id LIMIT 20` กลับเป็นต้นทุนก้อนใหญ่ที่สุดของทั้งระบบ ที่ 10M rows มันบังคับให้ planner เดินไล่ primary key แล้วจึงเข้าตาราง `numbers` ทีละแถว รวม 52,000 ครั้ง ใช้เวลา 16 ถึง 424 ms สำหรับ `123***` ผลการค้นหาไม่มีลำดับที่มีความหมายอยู่แล้ว จึงตัดออก และเวลาต่อ query ลดลงเหลือต่ำกว่าหนึ่งมิลลิวินาที อีกกฎคือ **อย่า materialize ผลลัพธ์ทั้งก้อน** เพราะ `**3***` เข้าเงื่อนไขเป็นล้านใบ แต่ผู้ใช้ต้องการแค่ยี่สิบใบ `LIMIT` จึงต้องไปถึงขั้น index scan เพื่อให้เวลาที่ได้หน้าแรกคงที่

## 5.4 Concurrency และการแจกของ

```
WITH picked AS (  
  SELECT id FROM tickets  
  WHERE status = 0 AND <pattern predicate>  
  LIMIT $2  
  FOR UPDATE SKIP LOCKED          -- ตัวรับประกัน  
) , upd AS (  
  UPDATE tickets t SET status = 1, reserved_by = $1, reserved_at = now()  
  FROM picked WHERE t.id = picked.id  
  RETURNING t.id, t.number  
) , ledger AS (  
  INSERT INTO allocations (ticket_id, user_id) SELECT id, $1 FROM upd  
)  
SELECT id, number FROM upd;
```

คำสั่งเดียวคือหนึ่ง transaction การเลือก การจอง และการบันทึกลง ledger จึงเกิดขึ้นครบทั้งหมดหรือไม่เกิดเลย `FOR UPDATE` ล็อกแถวที่เลือกไว้ ส่วน `SKIP LOCKED` ทำให้ transaction อื่นที่เข้ามาพร้อมกัน ข้ามแถวที่ถูกล็อกอยู่ไปหยิบแถวถัดไป แทนที่จะต่อคิวรอ ผล

ที่ตามมาคือข้อ ซึ่งเป็นเหตุผลที่วิธีนี้ดีกว่าทางเลือกอื่นใน §6.1 คือ ไม่มีทางที่ผู้ใช้สองคนจะได้ตัวใบเดียวกัน ไม่มี lock contention latency จึงไม่แย่งเมื่อคนแย่งกันมากขึ้น ไม่มีทางเกิด deadlock เพราะไม่มีใครรอใคร และไม่ต้องมีตัวประสานงานจากภายนอกเลย

### การแก้ปัญหาหวัหวั

มีรายละเอียดที่โผล่เฉพาะตอนโหลดต่อเนื่อง คือถ้าหวัของตามลำดับ `id` แถวที่ถูกจองไปแล้วจะยังกองอยู่หน้าสุดของลำดับ ทำให้การจองครั้งใหม่ต้องเดินผ่านของเก่าทั้งหมด วัดได้ว่าหลังจองไปเพียง 713 ใบ การจองหนึ่งครั้งต้องเดินผ่าน 68,453 แถว ใช้เวลา 145 ms และแย่งเรื่อยๆ

ทางแก้คือ**สุมเลขจริงที่อยู่ใน pattern** เช่น `***23` กลายเป็น `481723` แล้วยิง point lookup ถ้าพลาดค่อยลองใหม่สูงสุดสามครั้งก่อนจะถอยไปใช้การสแกน pattern ผู้ใช้ที่เข้ามาพร้อมกันจะกระจายตัวไปตาม  $10^{\text{wildcard}}$  เลข การชนกันจึงเกิดยาก และต้นทุนคงที่ไม่ว่าจะจองไปแล้วเท่าไร โดย `SKIP LOCKED` ยังเป็นตัวรับประกันความถูกต้องทั้งสองเส้นทาง ส่วนการสุมเป็นเพียงมาตรการด้านประสิทธิภาพ

### การหมดอายุของการจอง

การจองที่ไม่ถูกทำให้เสร็จต้องไม่ยึดของไว้ตลอดไป จึงมีงานเบื้องหลังคอยปล่อยของที่จองค้างเกินห้านาที ในระบบจริงงานนี้ควรอยู่ใน scheduled job แยก ไม่ใช่ใน process ของ API เพื่อให้ทำงานครั้งเดียวต่อรอบ ไม่ว่าจะมี replica กี่ตัว

หลักฐานที่วัดได้ว่ากลไกนี้ทำงานจริง คือการจอง 351,481 ใบภายใต้การแย่งกันโดยไม่มีตัวซ้ำอยู่ใน §8.1 บันทึกไว้เป็น ADR-003 และ ADR-004

# สิ่งที่วัดเทียบแล้ว และทำไมถึงแพ้

## 6.1 Storage และการประสานงาน

| สิ่งที่ต้องการ              | เหตุผลที่ POSTGRESQL ตอบได้                                                                                                                                                                   |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| แจกของโดยไม่ซ้ำ             | <code>FOR UPDATE SKIP LOCKED</code> เป็นเครื่องมือแจกงานที่ atomic และไม่มี deadlock ในตัว database เอง นี่คือปัจจัยชี้ขาด เพราะข้อที่ยากที่สุดของโจทย์กลายเป็นพีเจอร์บรรทัดเดียวของ database |
| ประสิทธิภาพการค้น wildcard  | expression index, partial index และ planner แบบคิดต้นทุน ทำให้ index พื้นที่ค้นหาหนึ่งล้านเลขได้หลายแบบ วัดได้ p95 56 ms ที่ 10M rows ภายใต้อุปกรณ์ 100 คนพร้อมกัน                            |
| ความถูกต้องเชิง transaction | การจองตัวกับการเขียน ledger อยู่ใน transaction เดียวกัน ซึ่งทำไม่ได้ถ้าแยก lock service ออกไป                                                                                                 |
| ดูแลง่าย                    | มี component เดียวที่ต้อง deploy, backup, monitor และทำความเข้าใจ พื้นที่รวมสำหรับสลาก 10 ล้านใบ พร้อม index ทั้งหมดอยู่ราว 1 GB                                                              |
| ยังโตต่อได้                 | โมเดลต้นทุนที่วัดได้ใน §7.2 แสดงว่า instance เดียวรับ 10,000 req/s ได้ที่ 39 % ของ 6 core                                                                                                     |

## ทางเลือกที่พิจารณาแล้วไม่เลือก

| ทางเลือก                                                         | เหตุผลที่ไม่เลือก                                                                                                                                                                                   |
|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ใส่ Redis เป็น cache ข้างหน้า                                    | เพิ่ม component และเพิ่มจุดที่พังได้ และสร้างปัญหาความไม่สอดคล้องของข้อมูลตรงจุดที่ความถูกต้องสำคัญที่สุด อีกทั้งการจองต้องเขียนลง database อยู่ดี cache จึงเป็นเจ้าของการตัดสินใจไม่ได้            |
| Redis distributed lock                                           | ต้องจัดการ lock expiry เอง ต้องรับมือกับ process ที่ตายขณะถือ lock และยังต้องเขียน database อยู่ดี ขณะที่ <code>SKIP LOCKED</code> ให้การรับประกันเดียวกันในระดับ transaction โดยไม่ต้องทำอะไรเพิ่ม |
| <code>SELECT FOR UPDATE</code> ที่ไม่มี <code>SKIP LOCKED</code> | ถูกต้องแต่ทำให้ทุกอย่างเรียงคิว ทุกคนต้องรอคนแรก latency พุงเป็นวินาทีที่มันมีคนแย่งกัน                                                                                                             |
| Optimistic locking ด้วย version column                           | เมื่อคนแย่ง pattern ยอดนิยมกันหนัก อัตราการ retry จะพุงจน throughput พังไปเอง                                                                                                                       |
| ล็อกในหน่วยความจำของ application                                 | ใช้ข้าม replica ไม่ได้ จึงพังทันทีที่ API ขยายออก ซึ่ง §7.2 ชี้ว่าเป็นเส้นทางขยายหลักของระบบนี้                                                                                                     |
| Elasticsearch หรือ search engine                                 | แก้ปัญหาค้นข้อความซึ่งไม่ใช่โจทย์นี้ ตัวเลขหลักเป็น key space ที่เล็กและตายตัว และมันไม่ได้ช่วยเรื่องการจองแบบ atomic ซึ่งเป็นความยากที่แท้จริง                                                     |

| ทางเลือก                          | เหตุผลที่ไม่เลือก                                                                                                                                        |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| คำนวณผลของทุก pattern ไว้ล่วงหน้า | มี pattern ที่เป็นไปได้ 10 <sup>6</sup> แบบ และผลลัพธ์บางอันมีเป็นล้านแถว ต้นทุนพื้นที่เก็บและการล้าง cache สูงกว่าต้นทุน query ที่พยายามจะเลี่ยงเสียอีก |

บันทึกไว้เป็น ADR-001 และ ADR-003

## 6.2 กลยุทธ์ index

การเสนอกกลยุทธ์เดียวโดยไม่มีตัวเปรียบเทียบ พิสูจน์ได้แค่ว่ามันทำงานได้ ไม่ได้พิสูจน์ว่ามันดี จึงสร้างและวัดทั้งสี่แบบบนข้อมูล 10 ล้านแถวชุดเดียวกัน

| กลยุทธ์                   | ทำอย่างไร                                                                                  | ขนาด INDEX     |
|---------------------------|--------------------------------------------------------------------------------------------|----------------|
| <b>0</b> Baseline         | <code>lpad(number::text,6,'0') LIKE '____23'</code>                                        | ไม่มีที่ใช้ได้ |
| <b>A</b> Expression index | สร้าง partial index แยกทีละหลักบน <code>tickets</code> แล้วหวังให้ planner รวมผลเอง        | 397 MB         |
| <b>B</b> Dimension table  | หาเลขที่เข้าเงื่อนไขจาก <code>numbers</code> ก่อน แล้ว join                                | 192 MB         |
| <b>C</b> B + prefix range | เหมือน B แต่หลักหน้าที่คงที่กลายเป็นช่วงบน key และ pattern ที่ระบุครบกลายเป็น point lookup | 192 MB         |

### ต้นทุนต่อ query เดียว หน่วยเป็นมิลลิวินาที

| PATTERN             | WILDCARD | 0    | A    | B    | C    |
|---------------------|----------|------|------|------|------|
| <code>123456</code> | 0        | 955  | 163  | 5.5  | 0.06 |
| <code>123***</code> | 3        | 2.6  | 19.9 | 3.0  | 0.12 |
| <code>****23</code> | 4        | 0.28 | 0.88 | 3.4  | 0.13 |
| <code>**3***</code> | 5        | 0.05 | 0.03 | 1.1  | 0.20 |
| <code>*****</code>  | 6        | 0.03 | 0.02 | 0.02 | 0.02 |

baseline ดูสุ่มในแถวล่างๆ เพียงเพราะ `LIMIT 20` ทำให้การสแกนหยุดได้เร็วเมื่อของที่เข้าเงื่อนไขมีเยอะ แถวที่วัดฝีมือจริงคือแถวบนสุด ซึ่งมีสลากเข้าเงื่อนไขราวสิบใบจากสิบล้านใบ และตรงนั้นแบบที่เลือก เร็วกว่าราว 15,000 เท่า

ภายใต้โหลด ผู้ใช้ 10 ถึง 100 คน บน API pod ขนาด 500m CPU ตัวเดียว

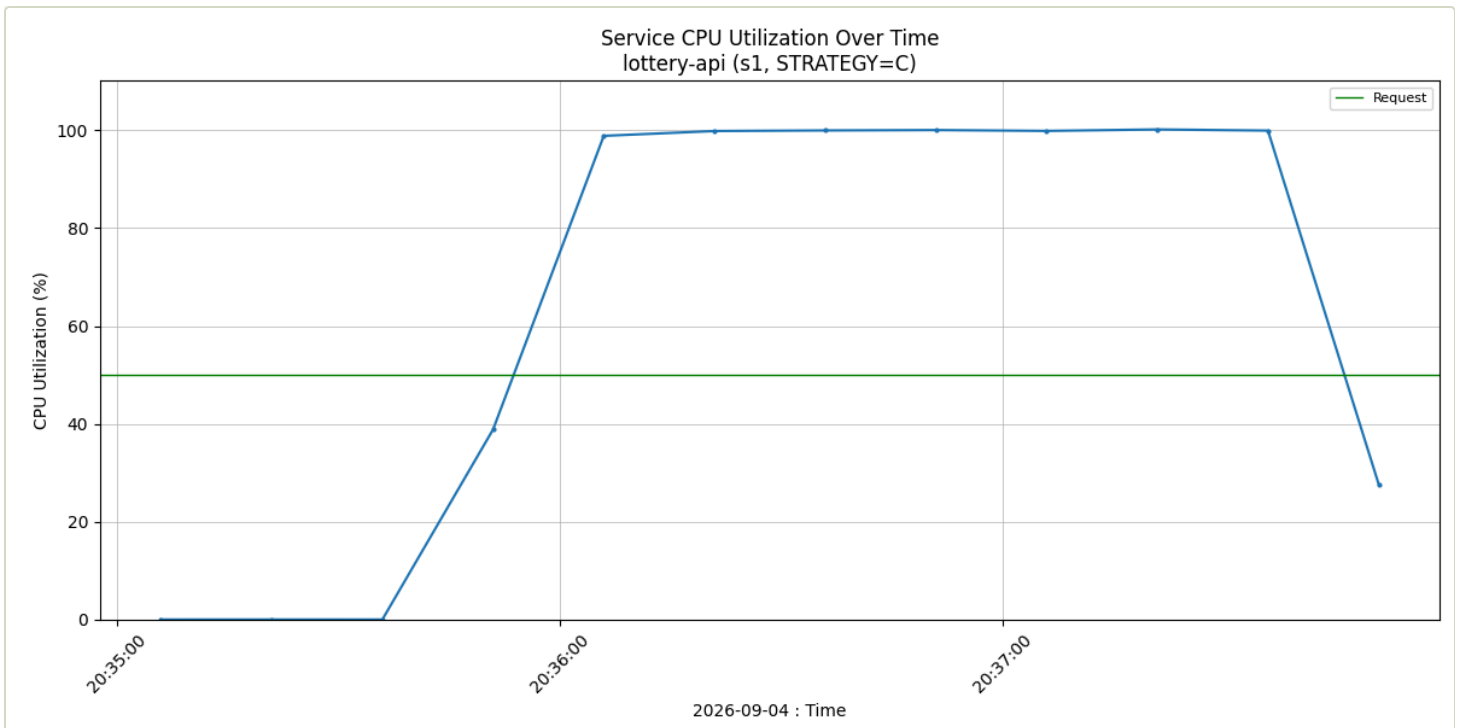
| กลยุทธ์ | P95 W0 | P95 W3 | P95 W4 | P95 W5 | P95 รวม | REQ/S | ล้มเหลว |
|---------|--------|--------|--------|--------|---------|-------|---------|
| C       | 56.3   | 55.9   | 55.5   | 55.6   | 55.8    | 5,346 | 0 %     |
| B       | 147.1  | 144.6  | 108.4  | 108.7  | 142.3   | 926   | 0 %     |
| A       | 6,718  | 5,162  | 4,859  | 4,860  | 5,972   | 22    | 0 %     |
| O       | 1,053  | 1,021  | 1,023  | 1,021  | 1,022   | 145   | 95.7 %  |

ภายใต้ Strategy C ค่า latency แบนราบเท่ากันทุกคลาส wildcard ซึ่งเป็นคุณสมบัติที่ §2.1 บอกว่าจำเป็น

แต่ละกลยุทธ์ใช้ทรัพยากรตรงไหน

| กลยุทธ์ | CPU ของ API   | CPU ของ POSTGRES | MEMORY ที่ POSTGRES เหลือ |
|---------|---------------|------------------|---------------------------|
| C       | 100 % ขนเพดาน | 88 %             | 203 MiB                   |
| B       | 24 %          | 101 % ขนเพดาน    | 279 MiB                   |
| A       | 1 %           | 101 % ขนเพดาน    | 8 MiB                     |
| O       | 1 %           | 102 % ขนเพดาน    | 568 MiB                   |

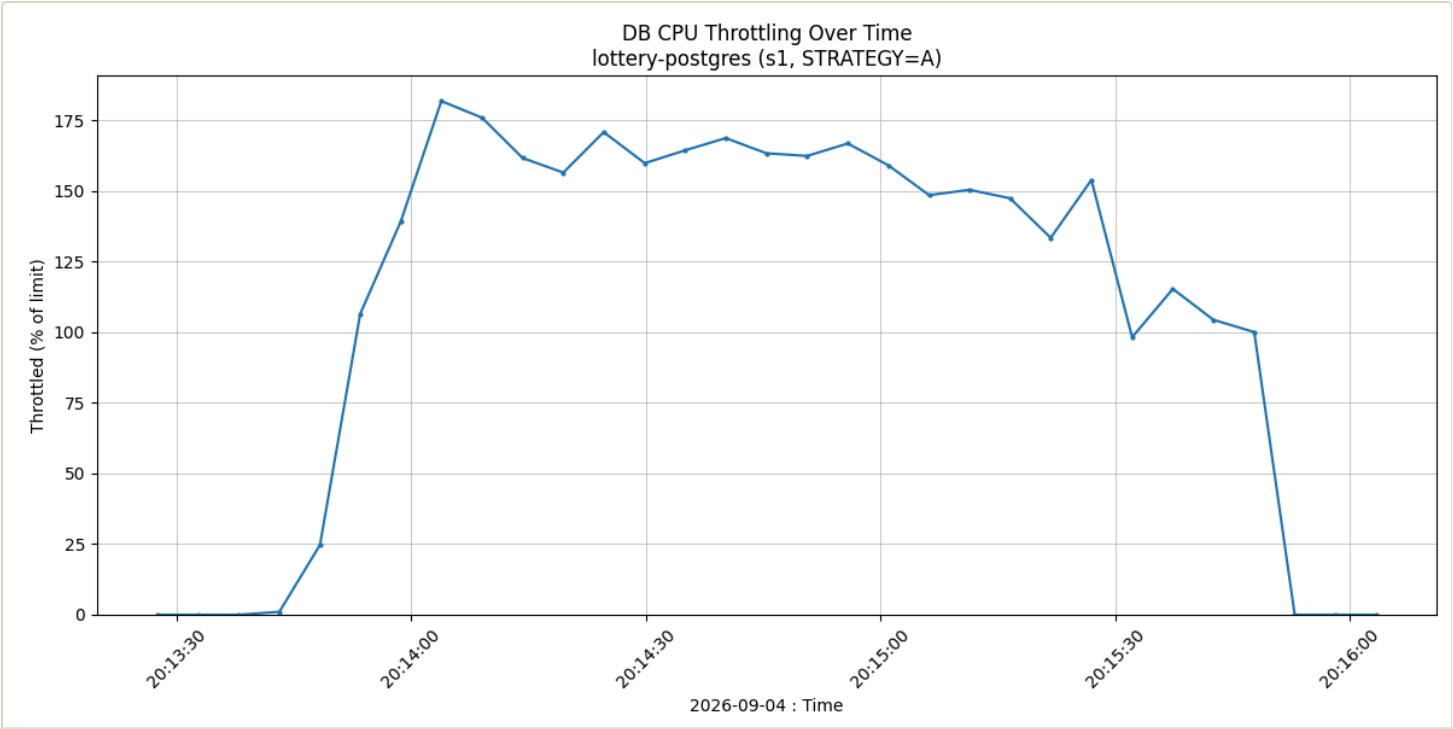
ตารางนี้สำคัญต่อการตัดสินใจออกแบบมากกว่าตาราง latency เสียอีก **Strategy C เป็นแบบเดียวที่ย้ายคอขวดออกจาก database ได้** เมื่อ Postgres ยังมีที่ว่างเหลือ การเพิ่ม throughput ทำได้ด้วยการเพิ่ม API replica ซึ่งถูกและแทบไม่มีขีดจำกัด ส่วน B กับ A นั้น database คือข้อจำกัด ต่อให้เพิ่ม API อีกก็ตัวก็ไม่ช่วย



ภายใต้ Strategy C ตัวที่เต็มก่อนคือ API pod ซึ่งเป็นรูปที่ต้องการ เพราะทรัพยากรที่เป็นข้อจำกัดคือตัวที่ทำสำเนาเพิ่มได้ เส้นสีเขียวคือค่า CPU request ของ pod

### ทำไม Strategy A ถึงล้มเหลว และทำไมข้อมูลนี้มีประโยชน์

Strategy A คือคำตอบที่คนส่วนใหญ่คิดถึงก่อน คือทำ index ทุกหลักแล้วให้ planner ตัดกันเอง แต่มันกลับเป็นตัวเลือกที่แย่ที่สุดจากที่วัดมา และการเข้าใจว่าทำไม คือสิ่งที่ทำให้การปฏิเสธรันมีน้ำหนัก ตัวเลขหลักเดียวคัดของทั้งได้แค่ 10 % คือ index บนหลักเดียวยังชี้ไปที่หนึ่งล้านแถวจากสิบล้านแถวอยู่ดี การเอา index แบบนี้หกดตัวมารวมกันจึงแพงกว่าที่ประหยัดได้ planner เลยเลือกใช้ตัวเดียวแล้ว filter ที่เหลือ



**Strategy A** ทำให้ database ถูกตัดโควตา CPU สูงสุดถึง 182 % ต่อวินาที พร้อมกับ memory ที่เหลือลดลงเหลือ 8 MiB จาก 1 GiB เพราะ index ขนาด 397 MB ไม่สามารถอยู่ร่วมกับข้อมูลที่ต้องใช้งานได้ บทเรียนนี้ใช้ได้ทั่วไป คือขนาดของ index กับความสามารถในการคัดของทั้งเป็นคนละเรื่องกัน และ index ที่คัดของทั้งไม่ได้มาก อาจแพงกว่าการสแกนทั้งตารางเสียอีก

บันทึกไว้เป็น ADR-002

6.3 Strategy D – วัดแล้วและตัดสินใจไม่ใช่

PostgreSQL 17 คั่น `= ANY(array)` ได้ในการไล่ B-tree รอบเดียว ซึ่งเปิดทางให้เขียน query อีกแบบที่เดิมทำไม่คุ้ม คือให้ API คำนวณเลขที่เข้าเงื่อนไขเองแล้วส่งเป็น `int[]` แทน การหาจาก subquery บนตาราง `numbers` เรียกว่า **Strategy D** ใช้ได้เฉพาะ pattern ที่มี wildcard ไม่เกินสามตัว คือ 1,000 คำ รว 4 KB ส่วนที่กว้างกว่านั้นถอยไปใช้ C

วัดด้วยสคริปต์เฉพาะที่มีคลาส `dd*d**` ซึ่งตัวเลขที่คงที่~~ไม่ได้~~อยู่ติดกันที่หลักหน้า Strategy C จึงไม่มีช่วง key ให้ใช้ ( `pg16-d1-*/` , `pg17-d1-*/` )

| เวอร์ชัน | STRATEGY | P95     | THROUGHPUT  | CPU ของ API | CPU ของ DB |
|----------|----------|---------|-------------|-------------|------------|
| PG16     | C        | 58.3 ms | 5,023 req/s | 100 %       | 72 %       |
| PG16     | D        | 39.1 ms | 6,051 req/s | 100 %       | 37 %       |

| เวอร์ชัน | STRATEGY | P95     | THROUGHPUT  | CPU ของ API | CPU ของ DB |
|----------|----------|---------|-------------|-------------|------------|
| PG17     | C        | 38.8 ms | 4,982 req/s | 100 %       | 73 %       |
| PG17     | D        | 33.7 ms | 6,047 req/s | 100 %       | 37 %       |

ผลออกมาสวนทางกับสมมติฐานที่เป็นเหตุผลของการทดสอบนี้ เดิมคาดว่า Strategy D ต้องพึ่ง optimization ของ 17 ถึงจะคุ้ม แต่กลับกลายเป็นว่ามันลด CPU ของ database ลงครึ่งหนึ่งทั้งสองเวอร์ชัน เพราะสิ่งที่ประหยัดได้มาจากการตัด join กับตาราง numbers ที่ไม่ได้มาจาก index scan แบบใหม่ และมีอีกสองข้อที่สำคัญกว่าหัวข้อหลัก คือ **การอัปเดตให้ latency เท่ากับการเขียน Strategy D** เพราะ C บน 17 ที่ 38.8 ms แทบไม่ต่างจาก D บน 16 ที่ 39.1 ms ซึ่งอย่างหนึ่งง่ายด้วยการขยับเวอร์ชัน อีกอย่างง่ายด้วยการเพิ่มเส้นทางโค้ดอีกชุด และ **สิ่งที่ D ให้จริงๆ คือ headroom ของ database ไม่ใช่ความเร็ว** เพราะ throughput ของมันเท่ากันทั้งสองเวอร์ชัน เนื่องจากที่ CPU ของ database เพียง 37 % มันติดที่ API และ database ที่เร็วขึ้นช่วยอะไรไม่ได้กับงานที่ไม่ได้รอ database อยู่แล้ว

**จึงตัดสินใจไม่ใช่ Strategy D** เพราะ §7.2 ระบุว่า database ใช้เพียง 39 % ของที่จัดสรรไว้ที่ 10,000 request ต่อวินาที การเพิ่ม headroom จึงเป็นการแก้ปัญหาที่ระบบนี้ยังไม่มี ขณะที่ D เพิ่มเส้นทางโค้ดอีกชุด ต้องจองหน่วยความจำสำหรับเลขถึง 1,000 ตัวต่อ request และเพิ่ม traffic อีก 4 KB ทุกครั้งที่ค้นด้วย pattern แคบ มันจะเป็นคำตอบที่ถูกต้องก็ต่อเมื่อ database กลายเป็นข้อจำกัดในอนาคต ซึ่งตอนนั้นผลวัดนี้บอกว่ามันจะลด CPU ของ database ลงได้ราวครึ่งหนึ่ง

6.4 เวอร์ชันของ PostgreSQL: วัตบน 16 แนะนำ 17

ตัวเลขทุกตัวใน §6.2 วัตบน PostgreSQL 16.13 ตารางจึงระบุ 16 หลังจากนั้นได้รับชุดเดียวกันข้ามบน 17.11 ภายใต้เงื่อนไขเดียวกันทุกอย่าง คือข้อมูล 10 ล้านแถวชุดเดิม profile แบบ baseline ที่มี API pod เดียวขนาด 500m คู่กับ database 2000m/1Gi และใช้สคริปต์ชุดเดียวกัน ทั้งสองฝั่งจึงเทียบกันได้ตรงๆ

| รายการทดสอบ                | PG 16.13 | PG 17.11 | เปลี่ยนแปลง | หลักฐาน            |
|----------------------------|----------|----------|-------------|--------------------|
| Strategy C ค้นหา p95       | 55.8 ms  | 41.9 ms  | -25 %       | s1-C/ → pg17-s1-C/ |
| Strategy C ค้นหา ค่าสูงสุด | 142 ms   | 86 ms    | -39 %       | ”                  |



| รายการทดสอบ                         | PG 16.13      | PG 17.11      | เปลี่ยนแปลง | หลักฐาน            |
|-------------------------------------|---------------|---------------|-------------|--------------------|
| Strategy C throughput               | 5,346 req/s   | 5,952 req/s   | +11 %       | "                  |
| การจองภายใต้ contention 200 VUs p95 | 73.8 ms       | 68.0 ms       | -8 %        | s2b/ → pg17-s2b/   |
| อัตราการจอง                         | 4,258/s       | 4,419/s       | +4 %        | "                  |
| ตัวที่ถูกแจกจ่ายภายใต้ contention   | 0 จาก 255,677 | 0 จาก 265,344 | เท่าเดิม    | 30-verify.txt      |
| Baseline LIKE อัตราที่ล้มเหลว       | 95.7 %        | 96.3 %        | เท่าเดิม    | s1-0/ → pg17-s1-0/ |

**ควรใช้เวอร์ชัน 17** การค้นหาได้ latency ดีขึ้น 25 % และ throughput เพิ่ม 11 % โดยไม่ต้องแก้ไขโค้ดเลย และการรับประกันว่าไม่แจกจ่ายไม่กระทบ เพราะการจอง 265,344 ใบภายใต้การแย่งกัน 200 ทางบน 17 ไม่มีตัวซ้ำเลยแม้แต่ใบเดียว เหมือนกับบน 16 ทุกประการ เนื่องจากการรับประกันนี้อาศัย SKIP LOCKED ไม่ได้อาศัยอะไรที่ผูกกับเวอร์ชัน

**แต่ไม่เปลี่ยนโครงสร้างอะไรเลย** ลำดับของ strategy เหมือนกันทั้งสองเวอร์ชัน และ baseline ยังล้มเหลว 96 % ภายใต้ผู้ใช้ 100 คนพร้อมกัน PostgreSQL 17 ทำให้ query เดียวๆ ของ baseline เร็วขึ้นถึง 83 % แต่ไม่ได้ทำให้พฤติกรรมภายใต้โหลดต่างไปเลย ซึ่งเป็นเรื่องตลกใจที่ชัดเจนว่า benchmark แบบ query เดียว กับ benchmark แบบมีผู้ใช้พร้อมกัน ตอบคนละคำถามกัน มีข้อความหนึ่งใน §6.2 ที่ต้องผ่อนลงตามนี้ คือ ความได้เปรียบเหนือ baseline ในเคสระบบครบหลัก วัดได้ 17,000 เท่าบน 16 และ 11,700 เท่าบน 17 เพราะ 17 ทำให้ baseline เร็วขึ้นมากกว่าที่ทำให้ Strategy C เร็วขึ้น ทั้งสองค่ามาจากการรัน EXPLAIN ครั้งเดียวและแกว่งไปมาในแต่ละรอบ ข้อความที่ยืนยันได้จริงจึงเป็น **สี่ order of magnitude ทั้งสองเวอร์ชัน**

ตัวเลขใน §6.2 คงไว้ตามที่วัดบน 16 ไม่ได้แก้เป็นของ 17 เพื่อให้ทุกตัวเลขในเอกสารมาจากแหล่งเดียวกัน ให้ถือว่าเป็นค่าที่ระมัดระวังไว้ก่อน บันทึกไว้เป็น ADR-007

# ข้อแลกเปลี่ยนด้านประสิทธิภาพ การขยาย การดูแล และการเฝ้าดู

## 7.1 ข้อแลกเปลี่ยนด้านประสิทธิภาพที่ยอมรับ

- มี **join** เพิ่มหนึ่งชั้น Strategy C อ่านสองตารางแทนที่จะเป็นตารางเดียว ต้นทุนที่วัดได้คือ เศษเสี้ยวของมิลลิวินาที เพราะตาราง numbers อยู่ใน cache ทั้งก้อน
- มีตารางที่ต้อง **seed** เพิ่ม ใช้คำสั่งเดียว รันครั้งเดียว
- **wildcard** ที่อยู่หน้าไม่ได้ประโยชน์จาก **prefix range** กรณี `****23` จะถอยไปใช้ index รายหลัก ซึ่งยังได้ 0.13 ms จึงไม่ใช่เส้นทางที่อ่อนแอ
- การจองแ่งลงเมื่อของใกล้หมด ตอน `****23` ถูกจองไป 96 % ค่า p95 ขึ้นจาก 74 ms เป็น 548 ms เพราะการสุมเลขไปเจอแต่ของที่ถูกจองแล้ว เรื่องนี้เป็นปัญหาระดับ product คือควรเลิกเสนอ pattern ที่ใกล้หมด ไม่ใช่ปัญหาของ database แต่ต้องออกแบบเผื่อไว้

## 7.2 การวางแผนขยาย

การทดสอบแบบขั้นบันไดที่ 1,000 / 2,000 / 3,000 / 4,000 request ต่อวินาที ช่วยแยกงาน พื้นหลังที่คงที่ออกจากต้นทุนที่เพิ่มขึ้นต่อหนึ่ง request

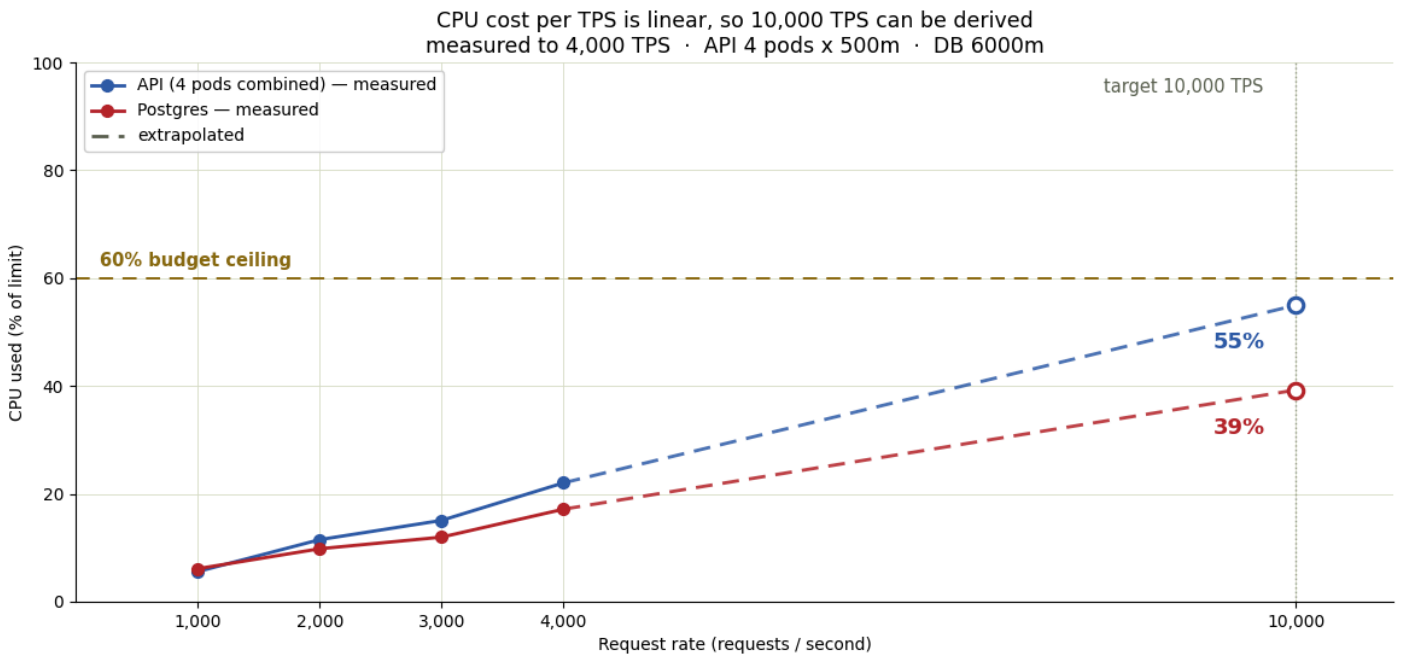
CPU ที่ใช้ = งานพื้นหลัง + ต้นทุนต่อ request × อัตราคำขอ

API = 1.9 m-core + 0.1098 m-core ต่อ req/s

Postgres = 146.1 m-core + 0.2209 m-core ต่อ req/s

เมื่อนำโมเดลนี้ไปใช้กับเป้าหมาย 10,000 request ต่อวินาที โดยให้ทั้งสองฝั่งใช้ทรัพยากรไม่เกิน 60 % ของ limit

| ส่วนประกอบ                              | REPLICAS | CPU LIMIT    | MEMORY LIMIT  | คาดว่าจะใช้ที่ 10,000 REQ/S   |
|-----------------------------------------|----------|--------------|---------------|-------------------------------|
| <b>API</b><br>ไม่มี state จึงขยายแนวนอน | 4        | 500m ต่อ pod | 128Mi ต่อ pod | <b>CPU 55 % • memory 14 %</b> |
| <b>PostgreSQL</b><br>ตัวเดียว           | 1        | 6,000m       | 3Gi           | <b>CPU 39 % • memory 49 %</b> |



เส้นทึบคือค่าที่วัดจริง เส้นประคือค่าที่คำนวณต่อ เส้นแนวนอนคืองบ 60 % ที่ 10,000 request ต่อวินาที ทั้งสองฝั่งยังอยู่ใต้เส้น การพล็อตเป็นเปอร์เซ็นต์ของ limit แทนที่จะเป็นค่า CPU ดิบ ทำให้ส่วนประกอบสองตัวที่ขนาดต่างกันใช้เส้นงบเดียวกันได้

มีสามเรื่องที่โมเดลซึ่งมองแต่ CPU จะมองข้าม เรื่องแรก **memory ของ database ไม่ได้โตเป็นเส้นตรงตามโหลด** แต่โตตามปริมาณข้อมูลที่ถูกดึงเข้า cache แล้วจึงแบนราบ โดยส่วนต่างของแต่ละขั้นคือ 256 → 103 → 66 MiB ซึ่งลู่อู่เข้าหาราว 1,510 MiB ถ้าลากเส้นตรงจะสั่ง memory เกินจำเป็นไปกว่าครึ่งกิกะไบต์ เรื่องที่สอง **connection pool ไม่ใช่ข้อจำกัด** เพราะจำนวนที่ต้องใช้พร้อมกันคือ อัตราคำขอ × เวลาต่อคำขอ = 10,000 × 1.19 ms ≈ 12 connection จากที่เตรียมไว้ 40 เรื่องที่สาม **ต้องขยาย API แนวนอน ไม่ใช่แนวตั้ง** เพราะ Go runtime ถูกตรึงไว้ที่หนึ่ง processor ต่อ pod การทำ pod ให้ใหญ่ขึ้นจึงไม่ช่วย แต่การเพิ่มจำนวน pod ช่วย และยังได้ความทนทานเพิ่มมาด้วย

เมื่อเกินราว 20,000 request ต่อวินาที database ตัวเดียวจะกลายเป็นข้อจำกัด ขั้นถัดไปตรงนั้นคือทำ read replica สำหรับการค้นหาลักษณะเดียว โดยให้การจองยังยิงเข้า primary เพราะ SKIP LOCKED ต้องการข้อมูลชุดที่เป็นต้นฉบับ

## 7.3 ความทนทานและการดูแลระบบ

**liveness** ห้ามขึ้นกับ **database** ตอนใช้ baseline strategy connection pool เต็มไปด้วย query ซ้ำๆ readiness probe จึงตอบไม่ทัน Kubernetes ถอด pod ออกจาก Service และ 19,134 จาก 19,990 request ล้มเหลว ซึ่งเป็นพฤติกรรมที่ถูกต้อง แต่ถ้า *liveness* probe ไปแตะ database ด้วย Kubernetes จะฆ่าและสร้าง pod ที่ยังดีอยู่ใหม่ระหว่างที่ database ซ้ำเปลี่ยนอาการซ้ำให้กลายเป็นระบบล่ม จึงต้องแยก `/healthz` ที่ตรวจแค่ process ออกจาก `/readyz` ที่ ping database

**ต้องบอก resource limit** ให้ **runtime** รู้ process ของ Go อ่านจำนวน core ของเครื่อง ไม่ใช่โควตาของ cgroup เว้นแต่จะสั่งเป็นอย่างอื่น จึงต้องตั้ง `GOMAXPROCS` และ `GOMEMLIMIT` ให้ชัดเจน ถ้าไม่ตั้ง runtime จะสร้าง thread มากเกินจริงและถูก OOM kill ในที่สุด

**ต้องดู CPU throttling** ไม่ใช่ดูแค่ **CPU** ที่ใช้ เพราะ throttling มองไม่เห็นในกราฟ CPU เฉลี่ย และมันคือกลไกที่ทำให้ Strategy A พัง นอกจากนี้ควรดู query plan ผ่าน `pg_stat_statements` ดู lock contention ผ่าน `pg_locks` และดูการใช้ทรัพยากรของแต่ละ pod เทียบกับ limit

ความเสถียรถูกยืนยันด้วยการรันต่อเนื่อง 10 นาทีแบบผสมทั้งค้นหา จอง และคืน ได้ 743,994 request ไม่มี error ไม่มี restart ไม่มี OOM kill และ memory ของ API แบนราบอยู่ที่ 27 MiB จาก limit 128 MiB

## 7.4 การเฝ้าดูระบบ

ดู query plan ผ่าน `pg_stat_statements` ดู lock contention ผ่าน `pg_locks` และ `pg_stat_activity` ดู CPU และ memory ของแต่ละ pod เทียบกับ limit และดู CPU throttling ซึ่งมองไม่เห็นในกราฟ CPU เฉลี่ย แต่คือกลไกที่ทำให้ Strategy A พัง (§6.2) observability stack เต็มรูปแบบ เป็นสิ่งที่ตั้งใจไม่ทำ (§3.2) แต่นี่คือสัญญาณที่มันจะต้องรองรับ

# ทุกข้อความ และตรวจสอบได้จากที่ไหน

## 8.1 หลักฐานจาก loadtest ว่าไม่แจกซ้ำ

ทดสอบสองรอบ แต่ละรอบมีผู้ใช้ 200 คนยิง pattern เดียวกันพร้อมกันเป็นเวลา 60 วินาที

|                                                                                                                                                                         | ****23 ของมีราว 100K ใบ | **3*** ของมีราว 1M ใบ |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|-----------------------|
| จองได้ใน 60 วินาที                                                                                                                                                      | 95,804                  | 255,677               |
| แถวใน ledger = ตัวที่ถูกจอง = id ที่ไม่ซ้ำ                                                                                                                              | 95,804                  | 255,677               |
| ตัวที่ถูกแจกสองครั้ง                                                                                                                                                    | 0                       | 0                     |
| ledger ที่ไม่ตรงกับสถานะตัว                                                                                                                                             | 0                       | 0                     |
| HTTP error                                                                                                                                                              | 0                       | 0                     |
| จอรวม 351,481 ใบ ซ้ำ 0 ใบ ตรวจสอบด้วย query เดียวที่ต้องคืนศูนย์แถว <code>SELECT ticket_id, count(*) FROM allocations GROUP BY ticket_id HAVING count(*) &gt; 1;</code> |                         |                       |

## 8.2 Automated test

เงื่อนไขเดียวกันถูกยืนยันใน `go test` กับ PostgreSQL จริง คือให้ 50 goroutine แย่งตัว 120 ใบ ซึ่งความต้องการมากกว่าของที่มี ผลคือทุกใบถูกแจกครั้งเดียวพอดี และเมื่อของหมดได้ผลลัพธ์ว่าง ไม่ใช่ error ชุดทดสอบรันกับทุกกลยุทธ์ index รวมทั้งที่ถูกตัดทิ้ง จึงแสดงได้ว่าการรับประกันความถูกต้องไม่ขึ้นกับเส้นทางการค้นหา

## 8.3 สารบัญหลักฐาน

| ข้อความ                                                 | แหล่งอ้างอิง                                                                                    |
|---------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| query plan และขนาด index                                | <code>docs/explain-k8s-10M.txt</code>                                                           |
| ต้นทุนของแบบที่ใช้ <code>ORDER BY</code> ซึ่งถูกตัดทิ้ง | <code>docs/explain-k8s-10M-orderby.txt</code>                                                   |
| การเปรียบเทียบกลยุทธ์ภายใต้โหลด                         | <code>docs/evidence/s1-C/</code> , <code>s1-B/</code> , <code>s1-A/</code> , <code>s1-0/</code> |

| ข้อความ                         | แหล่งอ้างอิง                                                                                                        |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------|
| การแจกของโดยไม่ซ้ำ              | <code>docs/evidence/s2/30-verify.txt</code> , <code>s2b/30-verify.txt</code>                                        |
| ความเสถียรตลอด 10 นาที          | <code>docs/evidence/s3/</code>                                                                                      |
| โมเดลการขยายไปถึง 10,000 req/s  | <code>docs/evidence/c1-capacity/50-capacity.json</code>                                                             |
| PostgreSQL 16 vs 17 query plans | <code>docs/evidence/pg16/explain.txt</code> , <code>pg17/explain.txt</code>                                         |
| PostgreSQL 17 under load        | <code>docs/evidence/pg17-s1-C/</code> , <code>pg17-s1-0/</code> , <code>pg17-s2b/</code>                            |
| Strategy D on both versions     | <code>docs/evidence/pg16-d1-C/</code> , <code>pg16-d1-D/</code> , <code>pg17-d1-C/</code> , <code>pg17-d1-D/</code> |
| บทวิเคราะห์หุ้กรอบการทดสอบ      | <code>docs/RESULTS.md</code>                                                                                        |

แต่ละชุดหลักฐานประกอบด้วย log ของตัวยิงโหลดแบบเต็ม, summary ทั้งแบบ JSON และ HTML, กราฟทรัพยากร, ข้อมูลดิบที่ใช้วาดกราฟ และผลตรวจ SQL ทั้งหมดรันซ้ำได้ด้วยคำสั่งเดียว

## 09 ความเสี่ยงและข้อจำกัด

### สิ่งที่การออกแบบนี้ยังไม่ได้พิสูจน์

- ตัวเลข **10,000 req/s** มาจากการคำนวณ ไม่ใช่การวัด อัตราสูงสุดที่วัดจริงคือ 4,000 req/s เป้าหมายจึงเป็นการลากเส้นตรงต่อไปอีก 2.5 เท่า ข้อมูลที่จุดสนับสนุนว่าเป็นเส้นตรงแต่ไม่ได้พิสูจน์ว่าจะจริงต่อไปเรื่อยๆ ก่อนขึ้นระบบจริงควรยิงจริงจากเครื่องแยก
- ยังเข้าใจพฤติกรรมตอนของไกล่หมดเพียงบางส่วน รู้แล้วว่า latency ของการจองจะแย่งเมื่อ pattern ไกล่ขายหมด แต่ยังไม่ได้หาว่าควรถอน pattern ออกจากการขายที่จุดไหน
- การวัดทั้งหมดมาจาก **node เดียว** ซึ่ง API, database และตัวยิงโหลดแย่ง CPU ชุดเดียวกัน ค่า throughput สัมบูรณ์บนเครื่องที่แยกกันจะต่างออกไป แต่การเปรียบเทียบระหว่างกลยุทธ์ ซึ่งเป็นฐานของข้อเสนอนี้ จะไม่เปลี่ยน
- ยังไม่ได้ทดสอบตอนระบบมีปัญหา ไม่มีการวัดใดครอบคลุมกรณี pod หาย การทำ rolling update หรือการ failover ของ database ซึ่งเป็นช่วงที่ความสามารถในการรับโหลดลดลง จึงควรเผื่อ replica เพิ่มอีกอย่างน้อยหนึ่งตัวจากที่คำนวณได้

- ยังไม่ได้ทดสอบตอนของถูกจองไปแล้วมาก การวัดทั้งหมดเริ่มจากของที่ว่างเต็มร้อย ถ้าสลาถูกขายไปแล้ว 80 % partial index จะเลิกลงแต่โอกาสที่การสุมเลขจะเจอของว่างก็ลดลงด้วย ทั้งสองผลต้องวัดจริง
- สิ่งที่ตั้งใจไม่รวมไว้ ได้แก่ authentication, payment, ระบบงวด, replication และ caching layer

10 ภาคผนวก

# บันทึกการตัดสินใจ คำศัพท์ และการเชื่อมกับเกณฑ์ให้คะแนน

## A. Architecture decision records

หนึ่งบันทึกต่อการหนึ่งการตัดสินใจ อยู่ใน adr/ แต่ละใบระบุบริบท การตัดสินใจ ทางเลือกที่ปฏิเสธ ผลที่ยอมรับ และชุดหลักฐานที่สนับสนุน

| ADR     | การตัดสินใจ                                                               | หัวข้อ     |
|---------|---------------------------------------------------------------------------|------------|
| ADR-001 | ใช้ PostgreSQL ตัวเดียว ไม่มี cache ไม่มี lock service                    | §6.1       |
| ADR-002 | หาเลขที่เข้า pattern บนตาราง numbers 1 ล้านแถวก่อน แล้ว join (Strategy C) | §5.3, §6.2 |
| ADR-003 | กันแจกซ้ำด้วย FOR UPDATE SKIP LOCKED ในคำสั่งเดียว                        | §5.4       |
| ADR-004 | จองด้วยการสุมเลขจริงก่อน แล้วค่อยถอยไปสแกน pattern                        | §5.4       |
| ADR-005 | ไม่ใช่ ORDER BY ในการค้นหา                                                | §5.3       |
| ADR-006 | liveness ห้ามแตะ database ส่วน readiness ต้องแตะ                          | §7.3       |
| ADR-007 | ใช้ PostgreSQL 17                                                         | §6.4       |
| ADR-008 | ไม่ใช่ Strategy D                                                         | §6.3       |

B. คำศัพท์

| คำศัพท์              | ความหมายในเอกสารนี้                                                                                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| p95                  | เวลาที่ 95 % ของ request เร็วกว่าค่านี้ ใช้แทนค่าเฉลี่ยเพราะค่าเฉลี่ยค่อนข้างต่ำๆ ที่ผู้ใช้รู้สึกได้จริง                                                                                        |
| VU                   | virtual user คือผู้ใช้จำลองหนึ่งคนในตัวยิงโหลด ที่ยิง request วนไปเรื่อยๆ 200 VUs คือจำลอง 200 คนพร้อมกัน                                                                                       |
| คลาส wildcard        | การจัดกลุ่ม pattern ตามจำนวน * (w0, w3, w4, w5) ต้องรายงาน latency แยกตามคลาส เพราะขนาดผลลัพธ์ต่างกันถึงทศออร์เดอร์                                                                             |
| SKIP LOCKED          | ตัวเลือกของ SELECT ... FOR UPDATE ที่สั่งให้ transaction ข้ามแถวที่ transaction อื่นถืออยู่ แทนที่จะรอ เป็นฐานของการรับประกันว่าไม่แจกซ้ำ                                                       |
| BitmapAnd            | วิธีที่ planner รวมผลจากหลาย index จะคุ้มก็ต่อเมื่อแต่ละ index คัดของทิ้งได้เยอะ index ของเลขหลักเดียวคัดได้แค่ 10 % จึงเป็นเหตุที่ Strategy A ล้มเหลว                                          |
| partial index        | index ที่จำกัดด้วยเงื่อนไข WHERE ในที่นี้คือ WHERE status = 0 จึงครอบคลุมเฉพาะสลาที่ว่าง และเลิกลงเมื่อของถูกขายไป                                                                              |
| working set          | memory ที่ใช้จริงโดยไม่นับ page cache ที่ระบบเรียกคืนได้ ตัวเลข memory ดิบของ container PostgreSQL จะอยู่ใกล้ 100 % ของ limit ตลอด เพราะมันเอาที่ว่างไปทำ cache working set คือตัวเลขที่ซื้อตรง |
| CPU throttling       | เวลาที่ container ถูกปฏิเสธ CPU เพราะใช้เกินโควตา มองไม่เห็นในกราฟ CPU เฉลี่ย ต้องวัดตรงจาก cgroup                                                                                              |
| liveness / readiness | liveness ถามว่า process ยังมีชีวิตไหม ถ้าไม่ผ่านจะถูก restart ส่วน readiness ถามว่ารับงานได้ไหม ถ้าไม่ผ่านจะถูกถอดจาก load balancer โดยไม่ restart                                              |

C. การเชื่อมกับเกณฑ์ให้คะแนนของโจทย์

| เกณฑ์                   | ตอบไว้ที่                                                                                |
|-------------------------|------------------------------------------------------------------------------------------|
| Feasibility             | §5.1 สถาปัตยกรรม, §5.2 โครงสร้างข้อมูล, §5.3 อัลกอริทึม ซึ่งสร้างขึ้นจริงและวัดผลแล้ว    |
| Performance             | §6.2 การเทียบสื่แบบที่ 10M rows และ §7.2 โมเดลการขยาย                                    |
| Correctness             | §5.4 การออกแบบเรื่อง concurrency และผลตรวจการจอง 351,481 ใบ                              |
| Real-world practicality | §6.1 การเลือก database และทางเลือกที่ไม่เลือก, §7.3 การดูแลระบบ, §9 ข้อจำกัด             |
| Creativity              | §2.1 พื้นที่ค้นหาหนึ่งล้าน, §5.3 การยุบหลักหน้าเป็นช่วงบน key, §5.4 การจองด้วยการสุ่มเลข |



เอกสารออกแบบสำหรับโจทย์ Lottery Search System ฉบับนี้เป็นภาษาไทย ส่วนฉบับภาษาอังกฤษ  
อยู่ที่ docs/DESIGN\_DOCUMENT.md และ docs/DESIGN\_DOCUMENT.html บทวิเคราะห์ประกอบ  
อยู่ใน docs/RESULTS.md และเอกสารวางแผนขยาย ส่วนข้อมูลดิบทั้งหมดอยู่ใน  
docs/evidence/